Applied Soft Computing Journal 186 (2026) 114253

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Large language model driven adaptive deep reinforcement learning with dynamic definition refinement for flexible job shop scheduling problem

Tzu-Yen Hong *, Kuan-Han Li

Department of Industrial Engineering and Management, National Taipei University of Technology, Taipei, Taiwan

HIGHLIGHTS

- A semantic-driven DRL framework LLM4A3C is proposed for solving FJSP.
- LLM dynamically redefines state and reward for enhanced RL adaptability.
- Closed-loop feedback improves policy robustness under dynamic conditions
- Results outperform baseline in makespan, tardiness, and sample efficiency.
- Ablation, sensitivity, and correlation analysis confirm the advantage of LLM4A3C.

ARTICLE INFO

Keywords:
Flexible Job Shop Scheduling Problem
Deep Reinforcement Learning
Large Language Model
Asynchronous Advantage Actor-Critic
Adaptive Learning

ABSTRACT

The flexible job shop scheduling problem (FJSP) remains a critical challenge in the transition to smart manufacturing due to its high complexity and the uncertainty of dynamic production environments. While reinforcement learning (RL) has emerged as a widely adopted approach for addressing FJSP, the definition of state representations and reward functions for RL remains a significant obstacle under changing system conditions. This study presents LLM4A3C, a Large Language Model for Reinforcement Learning (LLM4RL) framework that integrates a Large Language Model (LLM) with the Asynchronous Advantage Actor-Critic (A3C) algorithm to solve the FJSP. The proposed framework incorporates an LLM module to semantically adjust state representations and generate potential-based reward functions. A closed-loop feedback mechanism is designed to iteratively generate adaptive learning elements based on training results, enabling the DRL agent to dynamically respond to changes in the environment. Experimental results show that LLM4A3C significantly outperforms the baseline A3C in reducing makespan and total tardiness, while also improving sample efficiency and convergence speed. Furthermore, ablation and sensitivity analysis indicate the advantage of the proposed method and limitations of static feature and manual reward tuning. Indeed, the state signals also exhibit stronger alignment with actual return trends based on correlation analysis. The results validate that integrating LLMs into DRL architectures facilitates adaptive learning behavior in complex scheduling environments.

1. Introduction

Smart manufacturing emphasizes the intelligent and automated transformation of production processes through advanced technologies such as the Internet of Things (IoT), big data, cloud computing, and artificial intelligence (AI), aiming to enhance production efficiency and resource utilization. The Flexible Job-shop Scheduling Problem (FJSP) has been widely applied in fields such as mechanical manufacturing, remanufacturing, semiconductor fabrication, and automobile assembly [1]. With the increasing uncertainty and complexity in manufacturing environments, ensuring solution quality within finite computational time remains a critical concern [2,3]. Some previous studies have employed heuristic or metaheuristic algorithms to optimize objectives such as makespan, processing time, total tardiness, or energy consumption [4–6]. While heuristic algorithms are generally efficient and easy to implement, the results may get trapped in local optima. Moreover, the adaptability under changing system conditions is critical since the performances may degrade when applied to dynamic manufacturing environments [7]. Indeed, Reinforcement Learning (RL) has been increasingly applied to scheduling problems since the ability to learn

* Corresponding author.
E-mail address: tyhong@ntut.edu.tw (T.-Y. Hong).

https://doi.org/10.1016/j.asoc.2025.114253
Received 7 June 2025; Received in revised form 3 October 2025; Accepted 4 November 2025
Available online 11 November 2025
1568-4946/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

optimal actions at each state and respond dynamically to environmental changes based on feedback. For instance, Shiue et al. [8] adopted Q-Learning to address real-time scheduling (RTS) for optimizing various production performance indicators, while Wang [9] proposed a Weighted Q-Learning algorithm for solving the JSP to reduce tardiness. However, these conventional RL methods are limited to problems with relatively small state and action spaces. When applied to the large-scale problem with more complex environments, these methods may face significant computational burdens.

Thus, Deep Reinforcement Learning (DRL), which integrates Deep Neural Networks (DNNs) to handle larger state and action spaces of the RL framework, effectively addresses the limitations of conventional RL [10]. Moreover, DRL is capable of adapting to both discrete and continuous environments, enabling agents to learn and make decisions in complex scenarios [11]. Therefore, the application of DRL to FJSP using algorithms such as Deep Q-Networks (DQN), Proximal Policy Optimization (PPO), and Soft Actor-Critic (SAC) has demonstrated outperforming results in optimizing objectives such as makespan, total tardiness, and machine energy consumption [12–14]. However, the design of DRL models is often strongly dependent on the specific features of the scheduling problem. When the production environment, constraints, or optimization objectives change, the original definitions of the state representation and reward function may no longer be valid for generating reliable policies. In such cases, it is often necessary to redefine these components or retrain the model to maintain effectiveness under new conditions [15].

Due to the contextual understanding and reasoning capabilities of Large Language Models (LLMs) across various domains, LLMs are increasingly being integrated into RL frameworks to assist in reward function design and policy learning, particularly in addressing challenges such as sparse reward environments and dynamic task requirements [16]. For example, Zhang et al. [17] leveraged LLMs to compare preference trajectories to support reward shaping in RL. Sun et al. [18] proposed a dynamic reward evaluation method using LLMs to generate potential reward functions for diverse tasks. Qu et al. [19] enhanced the integration of LLMs in RL by automating reward function code generation and introducing a self-verification mechanism to reduce manual adjustments. Despite the growing interest in LLMs and their integration with RL, current applications remain largely confined to domains such as text processing and data management. The application of dynamic production scheduling to deal with the impacts of the rapid changes in manufacturing environments has seldom been explored.

To mitigate the need for frequent algorithm redesign under changing conditions, this study proposes an integration framework of LLMs and RL to deal with the FJSP to minimize makespan and total tardiness. The Asynchronous Advantage Actor-Critic (A3C) is used as the RL algorithm since the multi-threaded parallel learning mechanism accelerates the overall training process [20]. In particular, the proposed LLM for A3C (LLM4A3C) is developed by utilizing LLM to adjust RL state definitions and generate immediate reward values based on potential functions. Indeed, existing studies have demonstrated the effectiveness of LLM4RL mainly through reward shaping, such as latent reward inference, preference-based trajectory feedback, or task decomposition. These approaches primarily restrict LLMs in refining reward signals. Some extensions have explored the implementation of LLMs in designing action or state spaces, the related applications remain limited and often lack a systematic integration into the RL training loop. To address this gap, the present study introduces an LLM4A3C framework. The LLM acts as a dynamic agent that continuously adapts both reward functions and state representations based on training outcomes. A semantic-driven RL design mechanism enables the mapping from natural-language objectives and environment descriptions to adaptive policy design elements for RL agents. The closed-loop feedback structure further utilizes training results to inform the regeneration of learning definitions to enhance the adaptability of the learning process. By the proposed cross-module collaboration framework, DRL-based numeric evaluation can be coupled with LLM-driven semantic synthesis.

The contribution of this study can be summarized as follows. First, we develop an integration framework that leverages LLMs to enhance the adaptability of RL in solving the FJSP. The proposed semantic-driven RL framework dynamically redefines state and reward structures in response to variations in production environments, addressing the limitation that conventional RL models often lack flexibility when system environments change. By embedding LLM-generated semantic interpretations into the learning loop, the framework enables RL agents to adapt their decision-making process without requiring manual modification of the algorithmic structure. In addition, the framework incorporates a closed-loop refinement mechanism in which training outcomes from the RL process are iteratively analyzed and used to regenerate state–reward definitions through the LLM. Within this loop, a greedy policy is employed to guide sub-goal selection and refine definitions. Finally, by integrating with the A3C algorithm, the framework benefits from parallel training to improve computational efficiency and achieve generalization performance in various scheduling scenarios.

The remainder of this study is organized as follows: Section 2 reviews the related literature. Section 3 proposes the LLM4RL framework for FJSP. Section 4 presents the experimental results and analysis. Finally, the conclusions and future research are presented in Section 5.

2. Literature review

2.1. RL-based approach for solving the FJSP

FJSP has been widely investigated in previous studies. Many methods have been developed to address FJSP, with heuristic and metaheuristic algorithms emerging as the main solution strategies due to the effectiveness in handling multiple constraints [21]. For instance, Zandieh et al. [22] proposed the use of an improved imperialist competitive algorithm to minimize makespan for solving the FJSP with condition-based maintenance requirements. Li et al. [23] developed an improved artificial bee colony algorithm to address the FJSP with variable processing speed constraints, aiming to optimize the makespan, total carbon emissions, and machine loading. Guo et al. [24] employed a multi-level guided evolutionary algorithm to solve the FJSP with fuzzy processing times to optimize the makespan and the total workload of all machines. According to existing studies, heuristic algorithms can rapidly identify feasible solutions. Nevertheless, when production resources or process conditions change, metaheuristic algorithms often require re-execution or redesign, significantly increasing computational costs and processing time [7].

Indeed, RL has gained attention as an alternative approach for solving the FJSP, particularly in environments with high uncertainty. The training of RL learns from interaction with the scheduling environment, allowing it to adjust decisions based on real-time feedback and adapt to changes such as machine breakdowns, job priority shifts, or unforeseen resource constraints. For example, Li et al. [25] proposed utilizing the Q-learning method to optimally allocate production resources in an uncertain production environment with the objective of minimizing delays. Alexopoulos et al. [26] integrated Q-learning with Digital Twin (DT) systems to simulate dynamic scheduling and predict process changes, with the objective of minimizing production delays and reducing the computational cost associated with rescheduling. Furthermore, Serrano-Ruiz et al. [27] applied the Q-learning method in uncertain environments to optimize the minimization of makespan and machine idle time. However, the computational and spatial complexity limitations of Q-learning, some studies have employed state discretization techniques to simplify continuous spaces and reduce computational costs [28]. Nevertheless, such methods may result in reduced computational precision and a loss of model expressiveness [29].

Therefore, numerous studies have explored DRL to address the FJSP. For instance, Luo, S. [29] employed the DQN approach to address the challenge of maintaining the Q-table in a continuous state space. Their

method aims to minimize total operation delay while considering the insertion of new job orders. Similarly, Du et al. [12] employed the DQN method to address FJSP problems involving crane transportation and setup times, optimizing both total makespan and total energy consumption through a weighted optimization approach, thereby improving efficiency in high-dimensional state and action spaces. Johnson et al. [30] utilized the DDQN method for FJSP in robotic production lines, aiming to minimize total makespan while addressing the issue of Q-learning's overestimation of the value function. Additionally, Chang et al. [7] integrated a Soft ε-greedy strategy with DDQN in a dynamic FJSP environment to reduce early and late costs, while incorporating experience replay and target networks to enhance learning stability. Although DQN-based methods have been applied to FJSP, their reliance on deep neural networks to approximate the Q-function may result in unstable training in complex scheduling environments with sparse-reward settings.

In contrast, Actor-Critic-based DRL methods implement a division of roles between a policy network (Actor) and a value network (Critic), the separation improves learning stability by reducing the variance commonly associated with policy updates. Furthermore, these methods employ policy gradient techniques to determine action selection, enabling more flexible exploration of the action space and exhibiting stronger adaptability compared to DQN-based methods [31]. The A3C method proposed by Mnih et al. [20] employs asynchronous updates across multiple agents to reduce dependency on experience replay, thereby improving training efficiency and stability. Experimental results demonstrate that A3C outperforms DQN in continuous control tasks and high-dimensional environments, further substantiating the advantages of Actor-Critic-based approaches in specific application domains. Indeed, the Actor-Critic-based DRL methods have been increasingly applied to solving FJSP. For example, Zhang et al. [13] adopted the PPO algorithm to optimize FJSP with variable processing times, aiming to minimize total makespan. Ding et al. [32] extended this approach by introducing Multi-Agent PPO to minimize total makespan and delay time simultaneously. Gu et al. [33] utilized a multi-agent system framework combined with an Improved Genetic Planning PPO to optimize total makespan and energy consumption in FJSP. Ho et al. [34] proposed a residual scheduling method that leverages Graph Neural Networks (GNN) and the DRL approach to solve FJSP to minimize of total makespan. Wang et al. [35] introduced a Dual Attention Network (DAN) combined with an Actor-Critic framework to enhance scheduling efficiency and minimize total makespan in FJSP.

While DRL methods have been widely investigated in solving FJSP, several challenges remain in applications. The changes in problem settings, such as fluctuating processing times, dynamic machine availability, or the introduction of new jobs or workstations, may require manual reconfiguration or retraining of the RL agent. The system may be difficult to deploy in environments that demand frequent adaptation. Moreover, the states, actions, and rewards of RL need to be carefully defined. Designing these elements requires deep domain expertise and often involves trial-and-error tuning. When the scheduling context changes, these definitions may require to be redesigned. Static reward functions may fail to capture nuanced trade-offs between competing objectives, especially when the priorities shift over time. Therefore, this study proposes integrating LLMs as a semantic reasoning layer that can dynamically generate and adapt the state and reward definitions of RL based on problem context, system feedback, and scheduling objectives.

2.2. LLM for RL

LLMs have been widely applied in areas such as information retrieval, intelligent systems, and software engineering [36,37]. As research on their integration with reinforcement learning (RL) continues to grow, Pternea et al. [38] categorize current approaches into three main types: RL4LLM, where RL methods are used to improve the performance of LLMs in natural language processing tasks; LLM4RL, where LLMs assist RL training by helping define reward functions, formulate learning objectives, or provide semantic guidance for policy development; and RL+LLM, a parallel framework in which both components operate independently within the same system, supporting different functions without directly affecting each other's learning processes. Among these, LLM4RL offers the potential for improving the flexibility and adaptability of RL systems, especially in complex environments where manual design of learning components may be costly and inefficient. Indeed, LLM4RL which has been demonstrated in prior research as a key application area for generating and refining reward functions to enhance RL learning performance. For example, Zhang et al. [17] employed LLMs to compare different preference trajectories, generate feasible subgoals, and produce potential reward values for reward shaping. Sun et al. [18] further utilized LLMs to dynamically evaluate reward functions and employed three distinct preference trajectory feedback mechanisms to design potential reward values tailored to varying task requirements.

Recent studies have also explored various ways in which LLMs can support reinforcement learning by enriching the reward structure and reducing dependence on manual design. For instance, Pignatelli et al. [39] proposed decomposing complex task objectives using LLMs and generating auxiliary reward signals based on the decomposed subgoals. Qu et al. [19] introduced a method where LLMs automatically generate reward function code by leveraging domain-specific background knowledge, along with a built-in self-verification mechanism to ensure correctness. Nguyen et al. [40] focused on simulating user behavior, using evaluation metrics to produce additional reward values based on predicted user interactions. Tu et al. [41] developed a preference-based RL approach in which LLMs assign preference labels to trajectory pairs; these preferences are then used to generate improved training samples, effectively guiding the learning process through reward shaping. From these developments, two major approaches to LLM-based reward shaping in RL can be identified: (1) latent reward inference and updating, and (2) preference-based feedback using trajectory evaluation. Beyond reward shaping, recent studies have also extended the role of LLMs to other aspects of the RL framework, such as assisting in the design of state and action spaces, and even generating policies directly [42]. Based on these studies, this research adopts the LLM4RL approach by integrating large language models directly into the reinforcement learning process. In this study, the LLM is not used as a static support tool but rather as a dynamic agent that continuously adapts RL definitions, including state representations and reward structures, based on real-time training feedback. This integration enhances the system's ability to respond to changing conditions during training, improving the flexibility and generalizability of the RL framework. Table 1 illustrates the applications of LLM4RL methods.

3. Methodology

To address the challenges of dynamic production environments and the need for adaptive decision-making in FJSP, this study proposes an integrated framework featuring a LLM-driven definition refinement mechanism for RL, as illustrated in Fig. 1. The framework integrates a discrete event simulation (DES) system to simulate job routing, resource availability, and machine operations, serving as the interactive environment for the DRL agent. At the beginning of training, initial state, action, and reward definitions are formulated based on the problem structure and scheduling objectives to guide the early-stage DRL training through environment interaction. As training progresses, the accumulated results, such as learning trends, convergence status, and performance metrics like makespan and total tardiness, are summarized to an LLM-based definition generation module. This module leverages the LLM's semantic reasoning and generalization capabilities to propose refined representations of the problem. Two submodules are incorporated: one focuses on state definition refinement by analyzing performance data and generating candidate variable representations. The

Table 1
Applications of LLM4RL.

Work	Problem	Objective	Algorithm	LLM	LLM model
Zhang et al. [17]	Robot learning	Enhancing the efficiency of learning samples	PPO	Reward shaping	GPT−4
Sun et al. [18]	Robotic operation	Minimizing operational errors and failure counts	PPO	Reward shaping	GPT−4
Qu et al. [19]	Credit assignment problem	Minimizing reward prediction error	IPPO	Reward shaping	GPT−4o
Pignatelli et al. [39]	Credit assignment problem	Enhancing accuracy	PPO	Reward shaping	Llama−3
Nguyen et al. [40]	Query rewriting	Maximizing recommendation diversity and click-through rate	DPO	Reward shaping	Llama 3.1
Tu et al. [41]	Preference labels problem	Maximizing task completion objectives	SAC	Reward shaping	Gpt−4o-mini
Karine et al. [42]	Healthcare domain	Maximizing user behavior metrics and minimizing risk	TS	Action selection	Llama 3.1
This study	FJSP	Minimizing makespan and total tardiness	A3C	State representation、Reward shaping	Llama 3.2

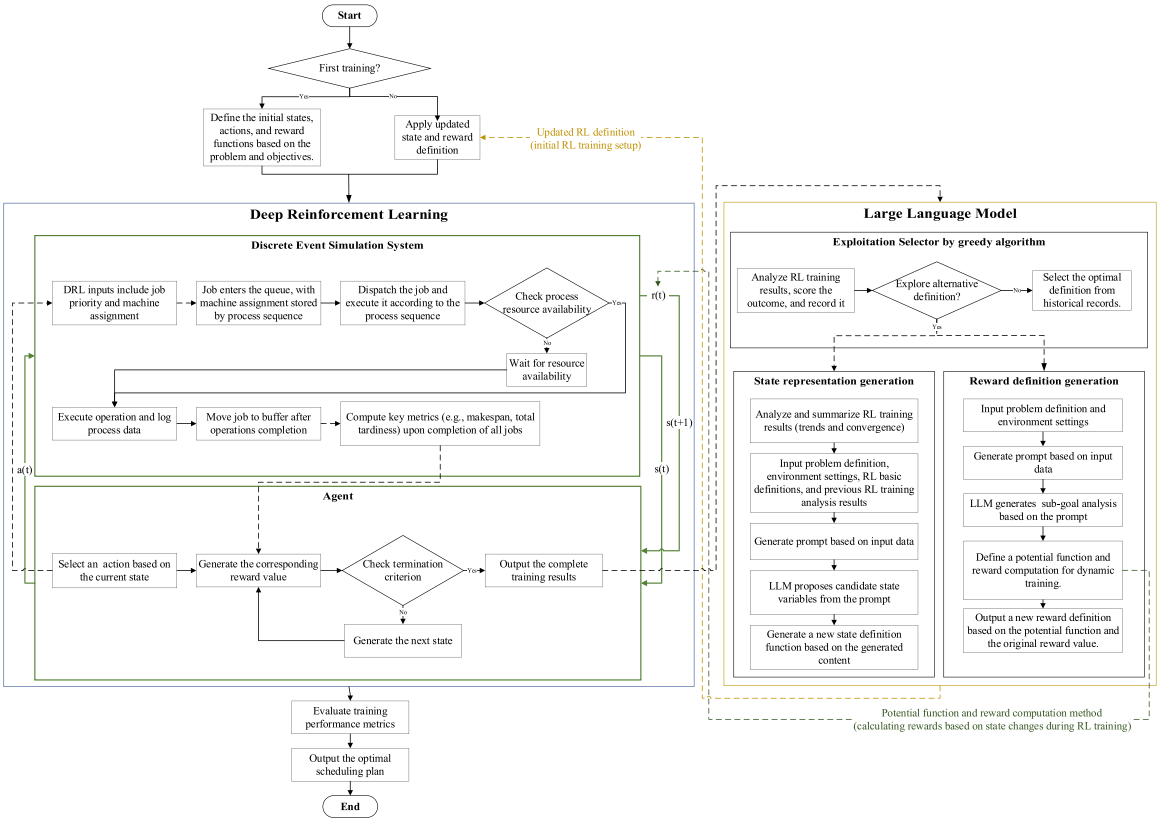


Fig. 1. Proposed research framework.

other constructs new reward computation mechanisms through prompt-based sub-goal identification and potential-based shaping. A greedy policy is adopted to decide whether to explore new definitions or retain the best-performing historical configurations. Once new definitions are generated, they are reintegrated into the DRL loop to enable the agent to re-learn under updated formulations.

Through this iterative interaction between the DRL training process and the LLM-guided definition refinement module, the proposed framework enables continuous adaptation of the internal learning structure to match the characteristics of the production environment. Training is terminated once a predefined stopping criterion is satisfied, at which point the optimized scheduling plan is generated and output for implementation.

3.1. Problem description

The FJSP related notation is listed as shown in Table 2. Given a set of jobs indexed by $i = 1,\ldots,n$, each job consists of J operations indexed by $j = 1,\ldots,J$, where each job follows a predefined processing sequence. The system includes $ws = 1,\ldots,WS$ workstations, each equipped with $k = 1,\ldots,m$ machines that have identical capabilities but potentially different processing times. Let p_{ijk} denote the processing time of the i-th operation of job i on machine k. All machines follow a single-processing constraint, meaning each can process only one job at a time, and no preemption is allowed. The objective is to optimize the assignment of jobs to machines to enhance overall scheduling efficiency. Eq. (1) defines the objective function, which aims to minimize the weighted sum of the makespan $C_{\max}$ and the total tardiness, where w_1 and w_2 denote the respective weights assigned to the makespan and the total tardiness.

Table 2
Notation for the FJSP.

Index	
Symbol	Description
i	Job index; i=1, …, n.
j	Operation index of a job; j=1, …, J.
ws	Workstation index; ws=1, …, WS
k	Machine index; k=1, …, m.
Parameter	
Symbol	Description
p_{ijk}	Processing time of operation j of job i on machine k.
d_i	Due date of job i.
w_1	Weight for makespan in the objective.
w_2	Weight for total tardiness in the objective.
M_{ij}	Feasible-machine list for operation j of job i
Variable	
Symbol	Description
x_{ijk}	1 if operation j of job i is processed on machine k; 0 otherwise.
c_i	Completion time of job i.
RP_i	The total remaining process time of job i
t	System time
$C_{\max}$	Makespan.
$\delta_i(t)$	Completion status of job i. 1 if the job is not yet completed at time t; 0 otherwise.
$N_{rem}(t)$	Number of unfinished jobs at time t.
$\tau_{tard}(t)$	Total tardiness at time t.
$P_{rem}(t)$	Average remaining processing time of all unfinished jobs at time t.
$P_{\min}(t)$	Shortest average processing time among feasible machines for the current job at time t.
$D_{avg}(t)$	Average due date of the remaining jobs at time t.
$D_{\min}(t)$	Earliest due date among remaining jobs at time t.
$D_{cur}(t)$	Due date of the current job at time t.
$\tau_{sys}(t)$	Current system time.
$I_k(t)$	Idle time of machine k at time t.
$T_{RP}(t)$	Total remaining processing time of selected job

Tardiness for job i is defined as $\max(0,\ c_i - d_i$, where c_i is the makespan of job i, and d_i is the due date of job i.

$$\min\left(w_1 \times C_{\max} + w_2 \times \sum_{i=1}^{n} \max(0,\quad c_i - d_i)\right) \tag{1}$$

Key characteristics of FJSP include parallel machine assignments for each operation, and the variability in product processing sequences should be considered while modeling the DES system. As illustrated in Fig. 2, jobs enter a centralized queue where their dispatching order is determined. The job sequencing and machine assignment decisions are made according to strategies derived from DRL agent. These strategies dynamically prioritize jobs and allocate them to appropriate machines based on system states and learned policies. The production process consists of a series of workstations, each equipped with a varying number of heterogeneous machines. The number of machines available, processing times, and product-specific routing sequences differ across workstations. As jobs progress through the workstations, each is processed by a selected machine following its predefined process route. Then, a data recording mechanism logs essential information, including the start and end times of operations, machine statuses, and resource availability. This information is captured through a "sink" component, which aggregates process-level records to calculate key performance metrics. These recorded data provide the performance metrics for evaluating of DRL across training episodes.

3.2. Deep reinforcement learning (DRL)

This study employs the Asynchronous Advantage Actor-Critic (A3C) algorithm as the DRL algorithm for solving the proposed FJSP. A3C updates are based on the policy gradient algorithm, utilizing the advantage function to evaluate the impact of actions on the current state. Indeed, A3C is employed due to the stable performance and relatively low computational overhead in environments with large state spaces such as the FJSP. The architecture with asynchronous updates provides a practical foundation for the proposed LLM4RL framework to balance the computational efficiency and adaptability. The detailed A3C process proposed by Mnih et al. [20] is as shown in Algorithm 1, in which advantage function, denotes as $A(s_t, a_t)$, is used to evaluate the advantage of selecting action a_t in a given state s_t. Here, $Q(s_t, a_t)$ represents the expected return of choosing action a_t in state s_t, while $V(s_t)$ denotes the value function of state s_t, as defined in Eq. (2). The policy update equation in A3C is based on the policy gradient algorithm, where the advantage function is utilized to update the policy parameters. The policy $\pi(a_t|s_t;\theta')$ defines the action a_t selection strategy in state s_t, parameterized by θ'. The advantage function estimate $A(s_t, a_t; \theta, \theta_v)$ is computed using the policy parameters θ and value function parameters θ_v. The update direction follows the gradient ascent of the advantage function, as expressed in Eq. (3).

$$\mathrm{A}(s_t, a_t) = Q(s_t, a_t) - V(s_t) \tag{2}$$

$$\nabla_{\theta'} \log \pi(a_t|s_t;\theta') A(s_t, a_t; \theta, \theta_v) \tag{3}$$

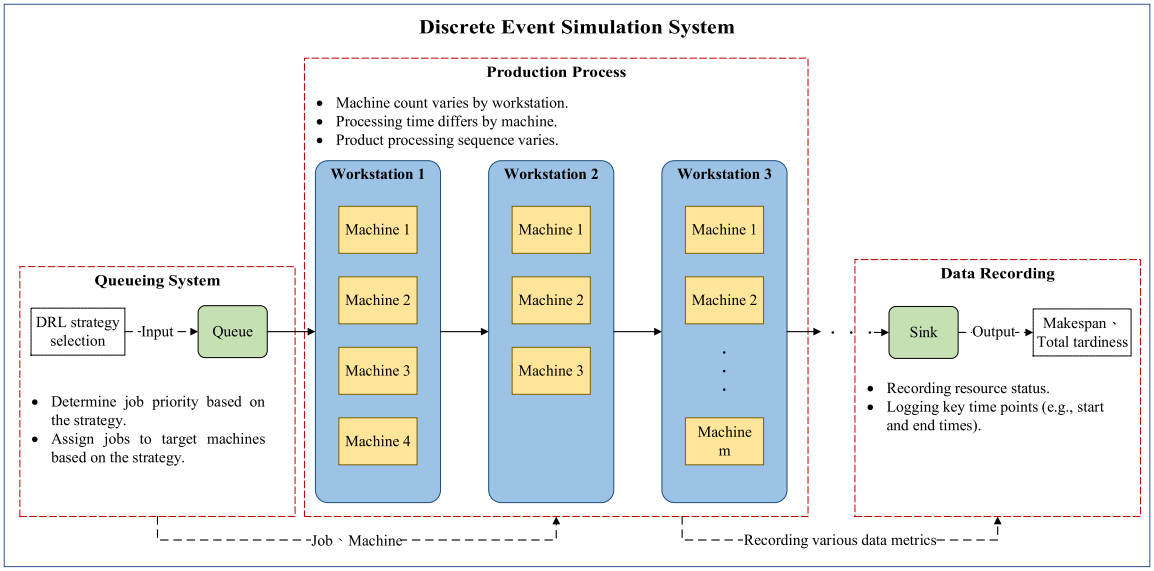


Fig. 2. Architecture of DES system for FJSP.

Algorithm 1. Asynchronous advantage actor-critic – pseudocode for each actor-learner thread

// Assume global shared parameter vectors θ and θ_v and global shared counter $T = 0$

// Assume thread-specific parameter vectors θ' and θ'_v

Initialize thread step counter $t \leftarrow 1$

repeat

Reset gradients: $d\theta \leftarrow 0$ and $d\theta_v \leftarrow 0$.

Synchronize thread-specific parameters $\theta' = \theta$ and $\theta'_v = \theta_v$

$t_{start} = t$

Get state s_t

repeat

Perform a_t according to policy $\pi(a_t|s_t;\theta')$

Receive reward r_t and new state s_{t+1}

$t \leftarrow t + 1$

$T \leftarrow T + 1$

until l terminal s_t **or** $t - t_{start} == t_{max}$

$R = \begin{cases} 0, \text{for terminal } s_t \\ V(s_t, \theta'_v), \text{for non terminal } s_t \text{ // Bootstrap from last state} \end{cases}$

for $i \in \{t-1, \ldots, t_{start}\}$ **do**

$R \leftarrow r_i + \gamma R$

Accumulate gradients wrt θ': $d\theta \leftarrow d\theta + \nabla_{\theta'} log\pi(a_i|s_i;\theta')\big(R - V(s_i;\theta'_v)\big)$

Accumulate gradients wrt θ'_v: $d\theta_v \leftarrow d\theta_v + \partial(R - V(s_i;\theta'_v))^2/\partial\theta'_v$

end for

Perform asynchronous update of θ using $d\theta$ and of θ_v using $d\theta_v$.

until $T > T_{max}$

to the machines with the shortest processing times. Eq. (11) represents the current system time $\tau_{sys}(t)$. Based on Eqs. (4) to (11), the state representation is formulated by a continuous state vector s_t as expressed in Eq. (12). The vector represents the number of unfinished jobs, the actual tardiness of the current job, the due date of the current job, the average remaining processing time of the unfinished jobs, the average due date of the remaining jobs, the minimum due date among the remaining jobs, the shortest average processing time of the machine for the current job, and the current system time.

3.2.1. State

The objective of the proposed FJSP is to simultaneously minimize both the makespan and the total tardiness. Therefore, this study defines a set of continuous state vectors to reflect the current system environment and assist the agent in decision-making. Eq. (4) defines the number of unfinished jobs $N_{rem}(t)$, in which δ_i denotes the completion status of job i. The value equals 1 if the job is not yet completed at time t. Eq. (5) represents the total tardiness $\tau_{tard}(t)$ of the jobs. Eq. (6) denotes the due date of current job in the system $D_{cur}(t)$. Eq. (7) calculates the average remaining processing time of all unfinished jobs $P_{rem}(t)$, in which M_{ij} denotes the set of the available machines for job i in operation j. Eq. (8) expresses the average due date of the remaining jobs $D_{avg}(t)$, and Eq. (9) identifies the earliest due date among the remaining jobs $D_{\min}(t)$. These equations are specifically designed for total tardiness optimization, helping the system analyze the workload of remaining jobs and assess due-date pressure. The agent allows for prioritizing urgent jobs or those with a higher risk of delay to reduce total tardiness. Eq. (10) represents the shortest average processing time of the machines $P_{\min}(t)$, and it is formulated for makespan minimization, guiding the agent to assign jobs

$$N_{rem}(t) = \sum_{i=1}^{n} \delta_i(t) \tag{4}$$

$$\tau_{tard}(t) = \sum_{i=1}^{n} \max(0, c_i - d_i) \tag{5}$$

$$D_{cur}(t) = d_i \tag{6}$$

$$P_{rem}(t) = \frac{1}{N_{rem}(t)} \sum_{i=1}^{n} \sum_{j=1}^{J} \delta_i(t) \left(\frac{\sum_{k \in M_{ij}} P_{ijk}}{|M_{ij}|} \right) \tag{7}$$

$$D_{avg}(t) = \frac{1}{N_{rem}(t)} \sum_{i=1}^{n} \sum_{j=1}^{J} \delta_i(t) \times d_i \tag{8}$$

$$D_{\min}(t) = \min_{i=1}\{d_i | \delta_i(t) = 1\} \tag{9}$$

$$P_{\min}(t) = \frac{1}{|M_{ij}|} \times \sum_{k \in M_{ij}} \min p_{ijk} \tag{10}$$

$$\tau_{sys}(t) = t \tag{11}$$

$$s_t = \begin{bmatrix} N_{rem}(t), \tau_{tard}(t), D_{cur}(t) P_{rem}(t), D_{avg}(t), D_{\min}(t), \\ P_{\min}(t), \tau_{sys}(t) \end{bmatrix} \tag{12}$$

3.2.2. Action

The agent's action selection involves choosing an appropriate job and assigning it to a suitable machine for processing based on its corresponding sequence. Therefore, this section summarizes the following four job selection rules (DPR-1 to DPR-4) and two machine assignment rules (DPR-5 to DPR-6) derived from dispatching rules proposed in previous studies. The rules are illustrated as follows.

(1) DPR-1: If no jobs are delayed, the job with the earliest due date is selected, as defined in Eq. (13). However, if tardiness existed, the job with the maximum tardiness is prioritized, as specified in Eq. (14). This rule aims to minimize total tardiness by prioritizing jobs based on their urgency.

$$\arg\min_{i \in n} d_i \tag{13}$$

$$\arg\max_{i \in n_{delay}} (c_i - d_i) \tag{14}$$

(2) DPR-2: The job with the highest risk of delay is selected, as defined in Eq. (15). This rule aims to minimize total tardiness by prioritizing jobs with the greatest delay risk, in which RP_i denotes the total remaining process time of job i.

$$\arg\min_{i \in n} (d_i - t) - RP_i \tag{15}$$

(3) DPR-3: The shortest processing time (SPT) is applied, where the job with the shortest processing time is prioritized, as defined in Eq. (16). This rule aims to minimize the makespan.

$$\arg\min_{i \in n} \sum_{k \in M_{ij}} p_{ijk} \tag{16}$$

(4) DPR-4: The critical ratio (CR) dispatching rule is applied, where jobs are selected based on their critical ratio, as defined in Eq. (17). This rule aims to minimize total tardiness.

$$\arg\min_{i \in n} \frac{d_i - t}{RP_i} \tag{17}$$

(5) DPR-5: The machine with the shortest processing time is selected, as defined in Eq. (18). This rule aims to minimize the makespan.

$$\arg\min_{k \in M_{ij}} P_{ijk} \tag{18}$$

(6) DPR-6: If idle machines are available, the machine with the longest idle time $I_k(t)$ (i.e., the time difference between the previous job's completion and the current system time) is selected, as defined in Eq. (19). Indeed, if no idle machines are available, the first available machine is assigned. This rule aims to minimize the makespan.

$$\arg\max_{k \in M_{ij}} I_k(t) \tag{19}$$

By combining the four jobs sequencing (JS) selection rules and two machine selection (MS) rules, a total of eight hybrid scheduling rules (HSR) are defined as the action selection space for the agent, as illustrated in Fig. 3.

3.2.3. Reward

The agent's immediate total reward is designed to compute the performances corresponding makespan and total tardiness. Eq. (21) represents the immediate reward for makespan $r_{makespan}$. The value is calculated based on two parts. First part is the difference between the average processing time of all machines selected in the next time step $(t+1)$ and the shortest average processing time in the current time step (t). The second part the difference in makespan between time steps $C_{\max}(t)$ and $C_{\max}(t-1)$. The average processing time of the machine

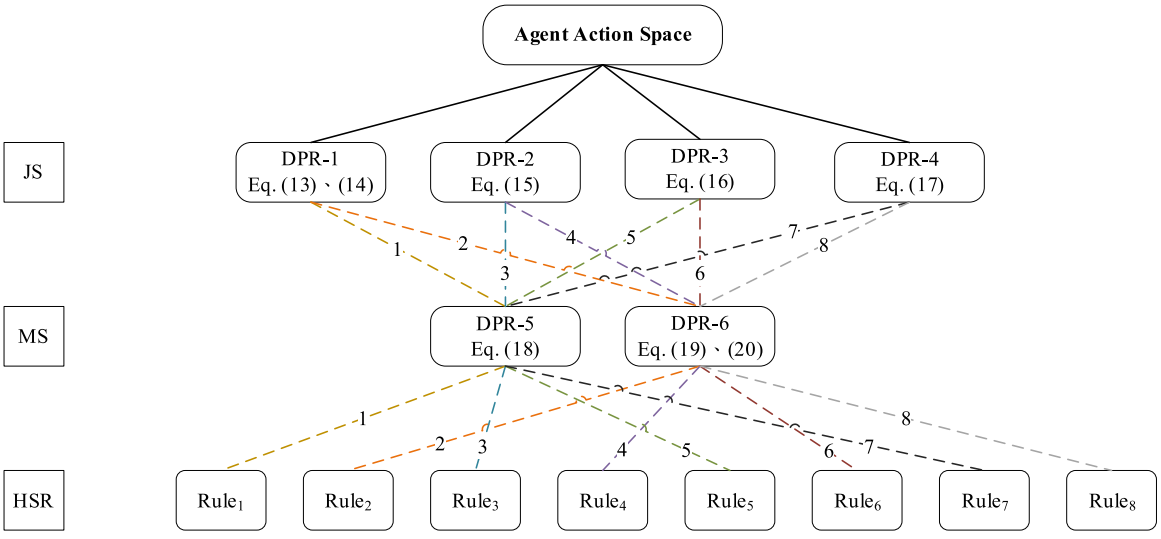


Fig. 3. Action selection space.

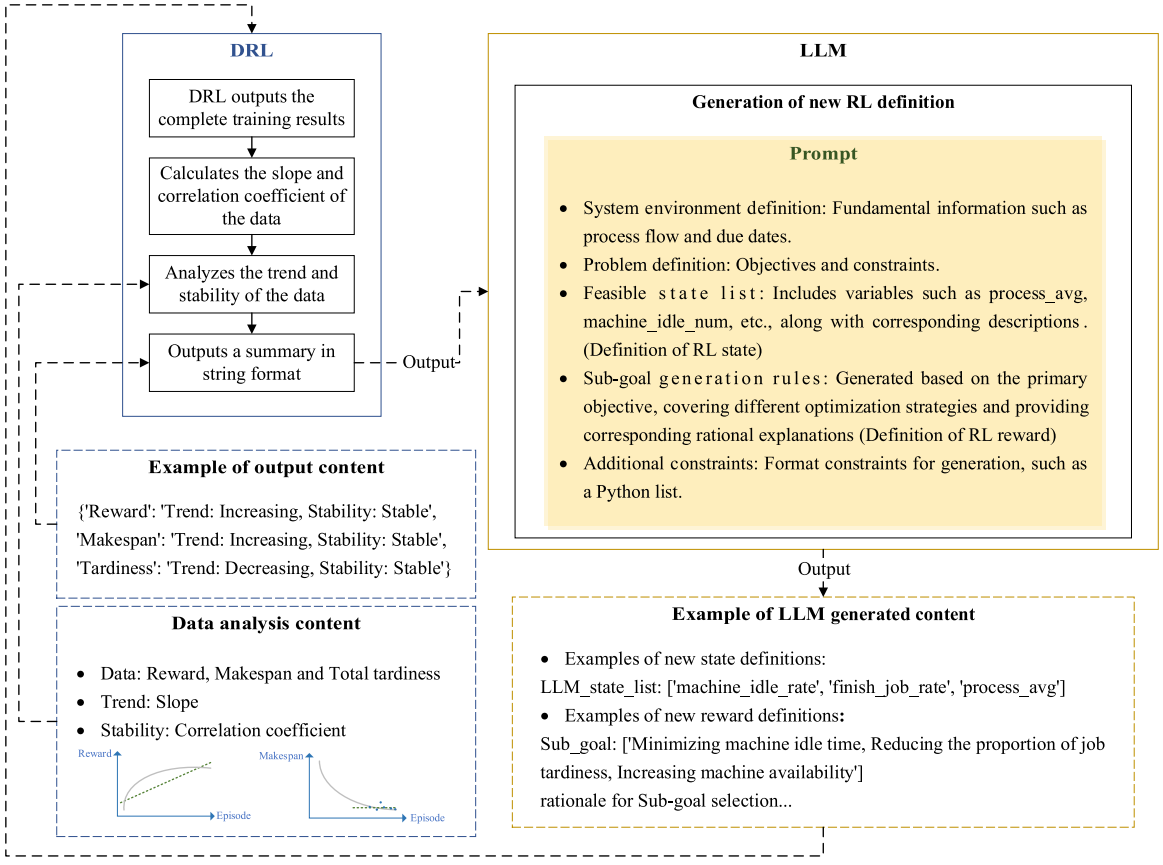


Fig. 4. Interaction process between DRL and LLM.

selected by the agent is defined as Eq. 10. The immediate makespan reward guides the agent to select appropriate jobs and assign them to machines with shorter processing times to improve makespan. Eq. (23) defines the immediate reward for tardiness $r_{tardiness}$ by evaluating the change between the current time step (t) and the next time step $(t+1)$. At time step $(t+1)$, if the remaining processing time before the due date at time step $(t+1)$ is negative, a negative reward is assigned. The remaining processing time at time $(t+1)$ can be expressed by Eq. (23) as $T_{RP}(t+1)$, where $d_i(t+1)$ denotes the due date of the selected job at the next time step. If the currently selected job has the earliest due date among all unfinished jobs, a higher positive reward is granted to encourage the agent to prioritize urgent jobs. Otherwise, the reward is calculated based on the difference between the average due date of the remaining jobs and the due date of the job selected by the agent at the next time step. The immediate tardiness reward guides the agent to prioritize jobs with nearer due dates and higher urgency, thereby reducing the overall total tardiness. To balance the optimization of makespan and total tardiness, Eq. (24) defines the overall immediate total reward r_t as a weighted combination of the two components. Here, w_α and w_β represent the weighting coefficients for makespan and tardiness, respectively, and are used to adjust their relative impact on the total reward. By leveraging both immediate reward design and the weighted combination approach, the agent can be effectively guided to learn appropriate scheduling decisions and achieve a well-balanced performance in weighted objective optimization. The detailed pseudocode for the reward calculation process is presented in Algorithm 2.

$$r_{makespan} = P_{\min}(t) - P_{\min}(t+1) + (C_{\max}(t-1) - C_{\max}(t)) \tag{21}$$

$$r_{tardiness} = \begin{cases} -T_{RP}(t+1), if\, T_{RP}(t+1) \le 0 \\ 100, if\, d_j(t+1) = D_{\min}(t) \text{and} T_{RP}(t+1) \ge 0 \\ \left(D_{avg}(t) - d_i(t+1)\right), \& otherwise \end{cases} \tag{22}$$

$$T_{RP}(t+1) = d_i(t+1) - t - RP_i \tag{23}$$

$$r_t = w_\alpha \times r_{makespan} + w_\beta \times r_{tardiness} \tag{24}$$

Algorithm 2. Reward Calculation Process

input: $s(t)$、$s(t+1)$

$s(t)$、$s(t+1)$: The state vector at time t and $t+1$.

output: The immediate reward r_t

procedure:

Calculate the average processing time of all selected machines for the next time step:

$$P_{\min}(t+1) = \frac{1}{|M_{pij}|} \times P_{ijk}$$

Calculate the remaining processing time until the due date at the next time step:

$T_{RP}(t+1) = d_i(t+1) - t - RP_i$

$r_{makespan} = P_{\min}(t) - P_{\min}(t+1) + \left(C_{max}(t-1) - C_{max}(t)\right)$

Calculate the immediate reward for the total tardiness time objective:

if $T_{RP}(t+1) < 0$ ***then***

    $r_{tardiness} = -T_{RP}(t+1)$

elif $d_i(t+1) = D_{min}(t)$ ***then***

    $r_{tardiness} = 100$

else

    $r_{tardiness} = \left(D_{avg}(t) - d_i(t+1)\right)$

Combine the rewards to calculate the immediate reward for the policy network:

$r_t = w_\alpha * r_{makespan} + w_\beta * r_{tardiness}$

return r_t

end;

3.3. Interaction between DRL and LLM

The interaction between DRL and LLM can be divided into input and output stages across several iterations. In the input stage, DRL training follows the original DRL-defined state and reward values if it is the first training iteration. However, for subsequent training iterations, the system adopts the updated state and reward definitions generated by the LLM to enhance learning performance, as shown in Fig. 1. In the output stage, the system outputs three key performance indicators after each DRL training iteration, including complete reward values, makespan,

and total tardiness of each episode in this iteration. The trends and stability of these indicators are analyzed using slope and correlation coefficient analysis, as illustrated in the data analysis content section of Fig. 4. The training results are converted into a string format, and a summary of the complete training outcomes is provided to the LLM, as shown in the example of output content in Fig. 4. Within the LLM component, contextual analysis is performed based on the given input and prompt, enabling the model to generate revised state representation and identify relevant sub-goals.

Within the LLM component, a structured prompt is constructed that includes (i) the system environment and problem definition, (ii) a feasible state list with predefined variables and descriptions, (iii) sub-goal generation rules linked to the optimization objectives, and (iv) additional constraints to ensure the correct output format. Based on the prompt, the LLM performs contextual analysis to generate refined state representations and potential sub-goals. The generated outputs are then validated by automatically mapping each new variable against the feasible-state library and by enforcing the required output. An illustration of this output is presented in the LLM-generated content example in Fig. 4. Indeed, the process also serves as the basis for the subsequent derivation of the immediate reward, which combines the computed results from the potential function with the original reward function, to guide the reinforcement learning agent's optimization behavior.

3.4. Large language model

This study proposes a framework that integrates LLMs with the A3C algorithm. By combining the semantic understanding and generation capabilities of LLMs with the powerful learning ability of A3C in high-dimensional spaces and complex environments, this framework enables the dynamic adjustment and automatic optimization of DRL states and reward values. The LLM-based approach in this study primarily involves the generation of new state definitions and new reward value definitions. The generation of new state definitions occurs outside the overall reinforcement learning framework, influencing each iteration of RL training based on the newly defined state representations. Moreover, new reward values are defined within the RL training framework to update rewards dynamically. This design allows the LLM to provide adaptive adjustments during the RL training process, improving overall learning performance and training efficiency.

3.4.1. Training methods for LLM

Since LLM-generated results may vary across iterations and cannot always guarantee consistency with the expected outcomes, this study incorporates an exploitation selector governed by a greedy policy external to the LLM framework. This mechanism is introduced to enhance the stability and reliability of the LLM-assisted training process. The Greedy Policy primarily operates during the generation of new state definitions, where it determines whether to adopt newly proposed definitions based on performance metrics. Although the LLM generates reward definitions dynamically by considering both current and subsequent states of RL, the reward definition process is also indirectly influenced by the greedy policy through its control over the state definition process.

During the exploration phase, the system leverages the LLM to generate a variety of diverse and feasible definitions, thereby promoting



Fig. 5. Integration of LLM Driven State Definition into DRL Training.

adaptability and solution diversity. In contrast, the exploitation phase selects the most effective definition from historical candidates, based on previously observed performance, to ensure steady progression toward an optimal strategy. In practice, the process begins by evaluating the training results from the preceding DRL cycle, where the trend of improvement and the stability of outcomes are quantified to form an evaluation score. This score is stored together with the associated state definition in a historical repository that serves as the memory for later exploitation. An ε-greedy rule is then applied to balance exploration and exploitation with probability ε the LLM is invoked to propose diverse new state-definition candidates, while with probability $1-\varepsilon$ the selector retrieves the best-performing definitions from the historical repository. The selected definitions are subsequently fed into the next DRL cycle, allowing the greedy policy to regulate reward definitions by controlling state definitions. By continuously evaluating, storing, and adopting state definitions, the proposed greedy policy improves both the efficiency of policy learning and the quality of final scheduling decisions. The detailed procedure of the greedy policy is presented in Algorithm 3.

Algorithm 3. LLM Exploitation selector with greedy policy

```
input: RL_training_results_data_summary, epsilon, state_history
procedure:
    # Evaluate RL training results based on trends and stability
    function evaluation_metric(RL_training_results_data_summary):
        # Assess the trend (e.g., increasing or decreasing) and stability of the results
        score = assess_trend_and_stability(RL_training_results_data_summary)
        return score
    # Compute evaluation score for the current training results
    current_score = evaluation_metric(RL_training_results_data_summary)
    # Store the evaluation result in state history
    update_state_history(state_history, RL_training_results_data_summary, current_score)
    # Exploration phase (ε-greedy strategy)
    if random() < epsilon, then
        # Use LLM to generate new state combinations
        LLM_state_list = generate_states_with_LLM(RL_training_results_data_summary)
    else:
    # Select the best states from historical data (greedy selection)
    LLM_state_list = select_best_states(state_history)
    return LLM_state_list
end;
```

constraints. These constraints are designed to help the LLM generate outputs that can be correctly interpreted and directly used by the DRL training process. The generated content is further structured to define new state variable functions that can be directly applied to the DRL training and optimization procedure. This design not only guarantees the validity and interpretability of the LLM-generated output but also ensures its seamless integration into the overall automated RL workflow.

In particular, the proposed procedure begins with DRL training based on an initial state definition. After a period of training, the results are summarized into a compact representation that captures the performance trends and stability of the agent. This summary is then provided to the LLM through a structured prompt. Leveraging semantic analysis of the training outcomes and task context, the LLM generates a revised set of state variables. As shown in the example in Fig. 5, the LLM preserves the original state variables while suggesting additional ones to capture system dynamics. Since the LLM output is produced in the form of a string-based list, an automated state integration is implemented to ensure compatibility with the DRL workflow. The module automatically recalculates the length of the continuous state vector, resets the dimensionality of the neural network input layer, and trains with the updated state representation to guarantee executability within the environment. The corresponding implementation functions are then invoked, ensuring that the DRL agent correctly computes the numerical values of each state during interaction with the environment.

3.4.2. Generation of new state definition

Based on the provided environment settings, the set of available state variables, and the summarized results from previous training episodes, the system constructs a structured prompt for the LLM. The design of this prompt including its format and content structure is illustrated in Fig. 4. In response to the prompt, the LLM generates new combinations of state variables, as demonstrated in the example LLM-generated content also as shown in the figure. To ensure compatibility with the downstream DRL automation process, the prompt is formulated with specific format

3.4.3. Generation of new reward definition

Based on the input environment settings, problem definitions, and summarized training results, a prompt is constructed for the LLM to generate corresponding sub-goals. The generation of these sub-goals is

constrained by the primary optimization strategy to enhance the adaptability of DRL in different scenarios. The potential function method $\varphi(s)$, as proposed by Zhang et al. [17], By computing the potential function values of the current state s and the next state s' at each DRL time step, the resulting value of the potential function, denoted as $F(s',s)$, can be derived, as defined in Eq. (26). In this equation, γ represents the discount factor, which regulates the impact range of rewards, while $\varphi(s)$ denotes the potential function value of state s. The computed result of the potential function $F(s',s)$ is subsequently integrated into the original reward function, thereby modifying and enhancing it, and further obtaining the immediate total reward value r', as shown in Eq. (27). By incorporating the results of the potential function into each DRL training iteration, this method provides additional learning signals to the agent, enabling it to learn more effective policies and thereby improve overall learning performance.

$$F(s',s) = \gamma\varphi(s') - \varphi(s) \tag{26}$$

$$r' = r + F(s',s) \tag{27}$$

Algorithm 4. Potential function calculation process

input: Environment setup

The LLM generates feasible sub-goals.

output: Sub-goal list and related descriptions

procedure:

s、s': The state vector at time t and $t+1$.

    # Define the potential function based on sub-goals:

    function compute_potential_function(s):

        # Sum contributions from all sub-goals

        $\varphi(s) = sub_{goal_1}(s) + sub_{goal_2}(s) + sub_{goal_3}(s) + \cdots sub_{goal_n}(s)$

        return $\varphi(s)$

while not check_RL_training_completion():

    # Compute the immediate reward based on the potential function values of the current and next states:

    $\varphi(s) = \text{compute_potential_function}(s)$

    $\varphi(s') = \text{compute_potential_function}(s')$

    # Compute the transition-based potential function result

    $F(s',s) = \gamma\varphi(s') - \varphi(s)$

    # Compute the immediate total reward

    $r' = r + F(s',s)$

end;

4. Experiments

Two different experiments were conducted to evaluate the effectiveness of the proposed method. First, this study implements the FJSP benchmark dataset derived from Hurink's instances as illustrated in Lei et al. [43] to evaluate performance. The optimality gap is measured by comparing the obtained results against the reported upper bound of each instance. Second, the experiments on the FJSP under multiple scenarios are conducted to validate performance. In this experiment, the Deep Q-Network (DQN) is introduced as a benchmark method. The results are compared across DQN, A3C, and the proposed LLM-driven extensions: LLM4DQN and LLM4A3C, to provide the evaluation against both general DRL and LLM-driven approaches. The setup of the parameters is based on prior works [29,32]. The total number of machines is set to 15. The number of available machines for each operation follows a uniform distribution $U[0, M]$, and the processing time of jobs on available machines is also uniformly distributed as U [1,20]. The due date for each job is determined as the product of a due date tightness factor f and the average total processing time across all operations of that job. Three test scenarios are considered: S1, S2, and S3. Each scenario is evaluated under three different job sizes: 15, 30, and 45. Scenario S1 represents the standard FJSP with normal production condition. Scenario S2 simulates an urgent order environment, where the due date tightness factor f is set to half of the value used in Scenario S1. Compared to S1, this results in significantly tighter due dates, effectively requiring jobs to be completed in half the time. Scenario S3 models a bottleneck machine condition, in which the processing time at a specific workstation is set to double that in Scenario S1. The processing times for the affected machines follow a uniform distribution U [1,20], representing a linear expansion of the original U [1,20] distribution to reflect the inefficiency of the bottleneck machines.

The computational experiments were implemented on a Windows 10 64-bit operating system equipped with 32 GB of RAM, Intel i7- 12700 CPU, and an NVIDIA RTX 3060 GPU with 12 GB of VRAM. For the A3C algorithm, the actor and critic networks contained 200 and 100 hidden units, respectively, with a global parameter update frequency of 5, batch size of 64, learning rate of 0.0001, discount factor γ equals 0.95, 1000

Table 3
Benchmarks on Hurink' Instances.

Instance		Lei et al. (2022)			DQN		A3C		LLM4DQN		LLM4A3C		UB	Percentage gap to Upper Bound			
		FIFO + EET	MOPNR + EET	PPO	Mean (std)	best	Mean (std)	best	Mean (std)	best	Mean (std)	best		DQN	A3C	LLM4DQN	LLM4A3C
15 ×	Vdata_la21	929	997	887	908.6 (29.8)	875.0	888.5 (21.4)	859.0	886.8 (35.3)	841.0	894.2 (31.9)	853.0	804	8.83 %	6.84 %	4.60 %	6.09 %
10	Vdata_la22	860	877	793	819.9 (27.4)	781.0	797.7 (9.8)	789.0	804.7 (15.3)	787.0	808.2 (15.6)	791.0	736	6.11 %	7.20 %	6.93 %	7.47 %
20 ×	Vdata_la26	1097	1164	1089	1102.7 (17.3)	1084.0	1076.5 (14.2)	1064.0	1084.9 (16.6)	1069.0	1071.2 (11.2)	1060.0	1054	2.85 %	0.94 %	1.42 %	0.57 %
10	Vdata_la27	1195	1219	1123	1145.3 (12.3)	1135.0	1120.4 (15.9)	1100.0	1133.0 (11.8)	1123.0	1117.8 (18.7)	1101.0	1084	4.70 %	1.47 %	3.60 %	1.57 %
30 ×	Vdata_la31	1577	1632	1561	1575.4 (20.6)	1557.0	1544.4 (49.5)	1527.0	1567.4 (23.8)	1530.0	1552.9 (38.1)	1525.0	1520	2.43 %	0.46 %	0.66 %	0.33 %
10	Vdata_la32	1717	1807	1693	1736.6 (40.3)	1690.0	1716.2 (61.7)	1671.0	1691.7 (40.4)	1671.0	1707.7 (51.1)	1669.0	1658	1.93 %	0.78 %	0.78 %	0.66 %
15 ×	Vdata_la36	1029	1113	985	1010.7 (21.8)	988.0	992.3 (44.2)	958.0	991.0 (40.4)	961.0	1021.9 (53.5)	978.0	948	4.22 %	1.05 %	1.37 %	3.16 %
15	Vdata_la37	1047	1165	1028	1045.8 (46.9)	1007.0	1016.9 (32.5)	993.0	1044.3 (33.6)	1011.0	1050.4 (35.4)	1002.0	986	2.13 %	0.71 %	2.54 %	1.62 %

Table 4
Optimal training metric results of different methods in various scenarios.

Job count	Scenario	DQN			A3C			LLM4DQN			LLM4A3C			Training duration(s)			
		Makespan	Total tardiness	Objective	Makespan	Total tardiness	Objective	Makespan	Total tardiness	Objective	Makespan	Total tardiness	Objective	DQN	A3C	LLM4DQN	LLM4A3C
15	S1	183.1 (10.7)	6.2 (13.5)	94.6 (10.2)	196.1 (18.7)	5.8 (16.4)	104.1 (14.2)	193.5 (9.2)	3.6 (11.3)	98.5 (18.0)	194.6 (14.2)	0.4 (0.9)	97.5 (7.4)	1765.0	514.1	2052.8	657.8
	S2	230.8 (18.3)	17.2 (23.1)	124.0 (15.4)	213.8 (20.3)	17.0 (20.1)	116.3 (10.7)	209.2 (17.9)	10.8 (17.3)	110.0 (2.2)	211.1 (12.2)	4.0 (9.9)	107.8 (5.0)	1772.0	508.5	2065.2	649.4
	S3	216.8 (17.2)	30.7 (40.9)	123.7 (25.3)	215.5 (14.3)	34.1 (30.2)	124.8 (19.8)	208.4 (9.5)	2.0 (3.7)	105.2 (6.5)	203.8 (26.5)	3.0 (6.0)	103.4 (14.5)	1763.4	498.7	1956.1	687.1
30	S1	227.6 (19.1)	6.5 (14.4)	117.4 (8.1)	222.2 (31.4)	10.3 (14.1)	116.2 (20.8)	205.5 (30.2)	7.3 (15.7)	106.4 (20.0)	198.4 (28.7)	3.6 (10.1)	101.0 (14.4)	5339.2	1050.6	5382.6	1072.5
	S2	261.1 (64.4)	27.6 (20.0)	144.5 (36.8)	265.8 (43.1)	33.2 (27.4)	149.5 (31.9)	239.8 (24.8)	8.8 (7.0)	124.3 (15.5)	230.0 (40.3)	17.3 (20.5)	123.7 (27.9)	6190.4	1067.4	6500.8	1077.2
	S3	258.2 (38.1)	30.1 (59.9)	144.1 (31.1)	264.0 (42.3)	38.5 (50.9)	151.3 (35.8)	245.6 (28.7)	19.8 (24.4)	132.7 (22.9)	253.9 (35.6)	15.4 (26.2)	134.7 (17.4)	6646.2	1056.5	7008.6	1375.2
45	S1	313.0 (49.3)	37.0 (42.7)	175.0 (44.4)	309.5 (55.8)	29.4 (49.7)	169.5 (49.7)	309.9 (20.4)	18.6 (48.8)	164.2 (39.6)	310.8 (31.7)	19.2 (24.1)	165.0 (27.2)	6756.3	1563.0	7343.0	1598.5
	S2	322.4 (56.7)	40.1 (56.0)	181.2 (49.0)	304.0 (60.5)	38.3 (59.1)	171.2 (56.2)	300.2 (39.7)	26.6 (51.6)	163.4 (43.5)	284.6 (42.4)	16.3 (43.7)	150.5 (41.6)	6833.4	1570.4	7802.4	1575.5
	S3	339.7 (61.6)	71.5 (98.1)	205.6 (71.7)	331.6 (73.3)	47.7 (66.3)	189.7 (64.8)	296.2 (29.7)	43.2 (73.6)	169.7 (50.9)	293.0 (39.5)	30.1 (41.7)	161.6 (38.6)	6899.1	1558.2	8189.8	2149.8

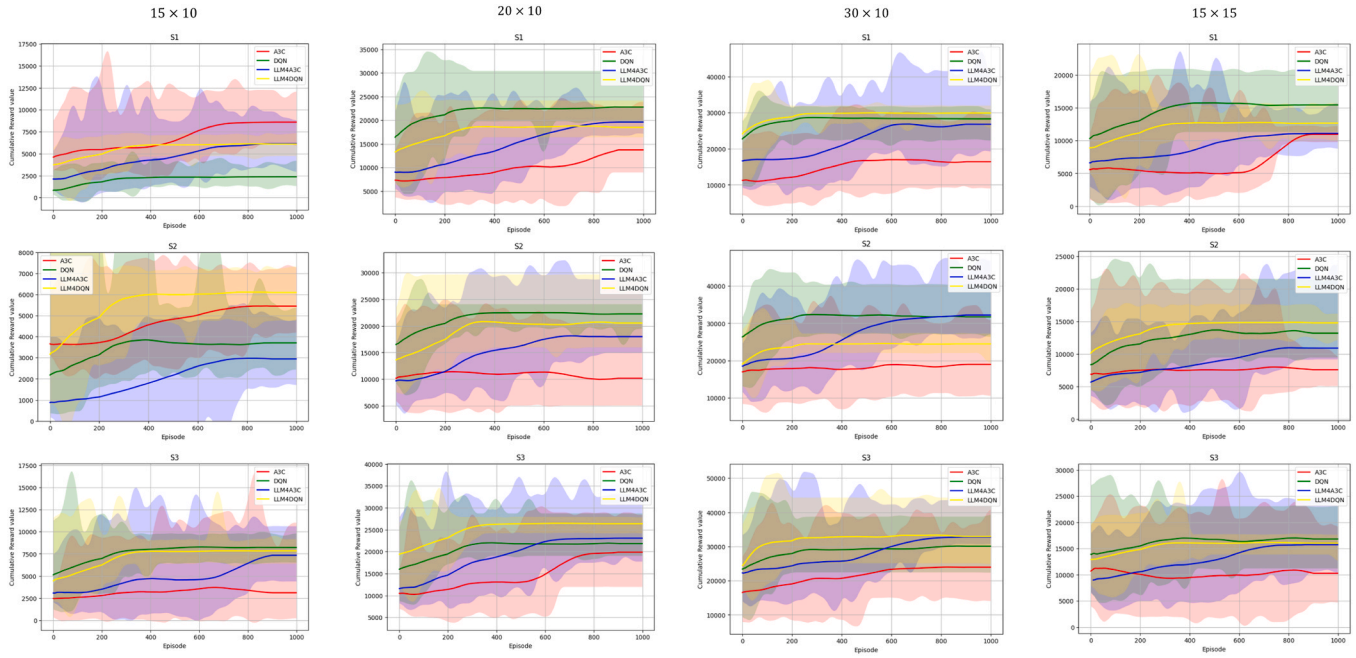


Fig. 6. Cumulative rewards in S1, S2 and S3.

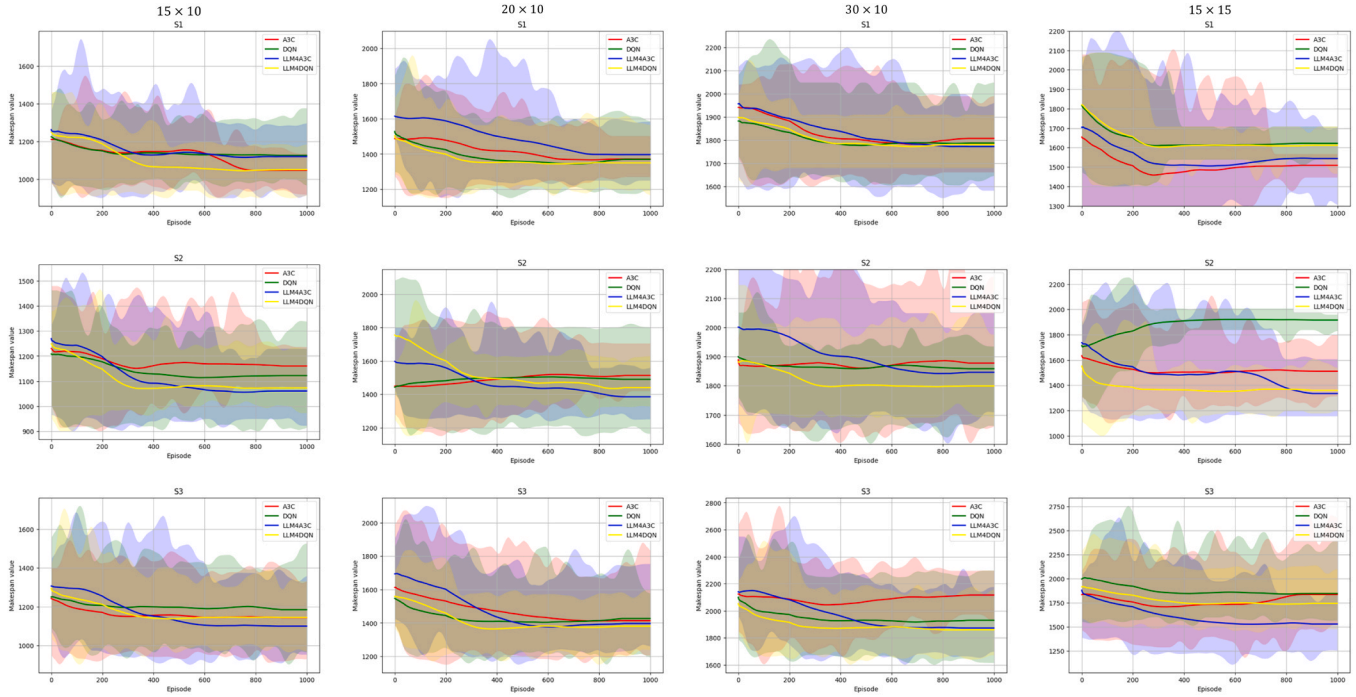


Fig. 7. Makespan in S1, S2 and S3.

training episodes, and 4 parallel processes. For the DQN algorithm, the hidden layer size was set to 128, the target network was updated every 100 steps, and the replay buffer size was 5000, while the batch size, learning rate, discount factor, and number of iterations were identical to those of A3C.

4.1. Benchmarks with Hurink's instances

The benchmarks in this section were conducted using the dataset derived from Hurink's instances, including problem sizes of 15 × 10, 20 × 10, 30 × 10, and 15 × 15. The proposed methods were compared with the PPO algorithm and the dispatching rules such as FIFO+EET and MOPNR+EET as sown in Lei et al. [43]. Moreover, the RL baselines, including DQN and A3C, as well as the LLM4DQN, are also included as benchmarks. The results as shown in Table 3 demonstrate the LLM-driven methods obtain smaller performance gaps of to the upper bound compared with the corresponding RL baselines. In the 20 × 10 and 30 × 10 instances, the best results produced by LLM4DQN and LLM4A3C were very close to the upper bound. A similar result can be obtained for 15× 10 and 15× 15, where the LLM-driven variants delivered the best solutions among all tested methods across various problem sizes. When compared with the best-known solutions reported

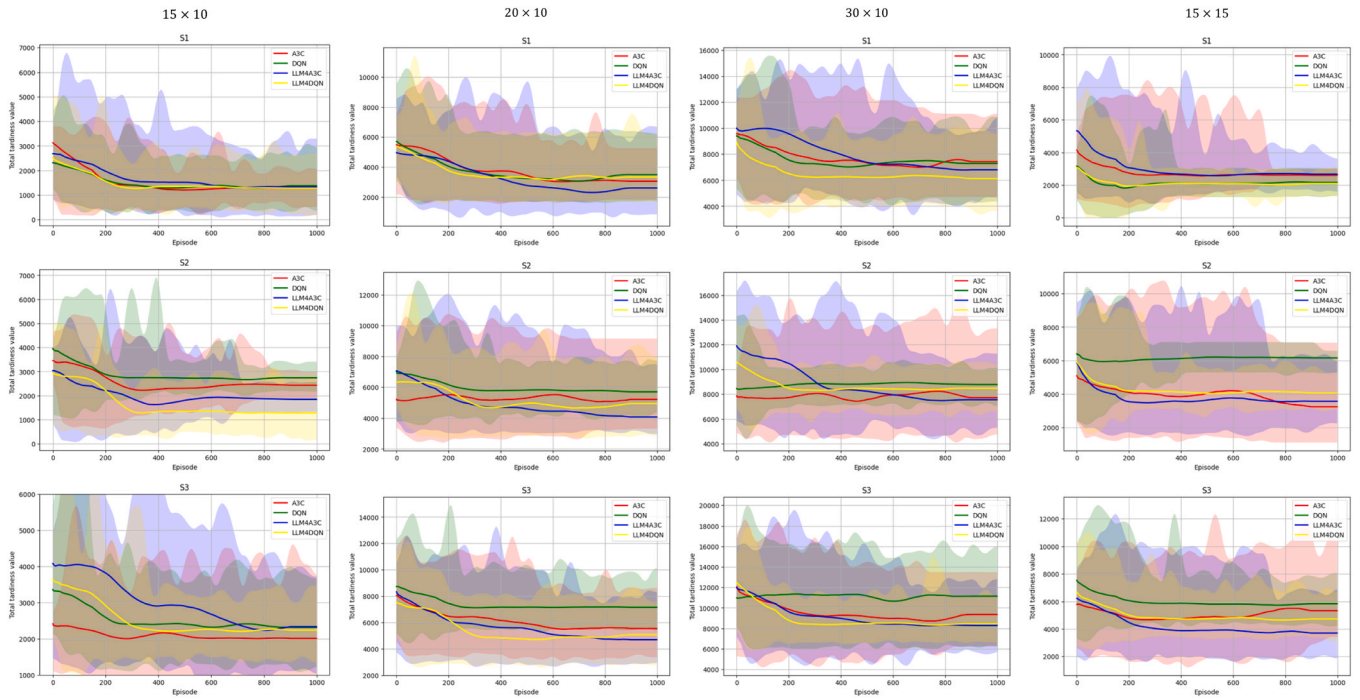


Fig. 8. Total tardiness in S1, S2 and S3.

Table 5
Sensitivity analysis of weights.

Weight	S2				S3			
	Makespan (std)	Total tardiness (std)	Gap with LLM4A3C		Makespan (std)	Total tardiness (std)	Gap with LLM4A3C	
			Makespan	Total tardiness			Makespan	Total tardiness
1:9	223.2 (18.1)	14.7 (20.2)	5.42 %	72.79 %	220.3 (32.7)	42.0 (55.8)	7.49 %	92.86 %
2:8	220.5 (16.1)	39.4 (42.4)	4.26 %	89.85 %	216.5 (27.5)	39.3 (49.2)	5.87 %	92.37 %
3:7	213.6 (23.8)	51.8 (46.8)	1.17 %	92.28 %	222.7 (7.7)	39.7 (57.2)	8.49 %	92.44 %
4:6	218.0 (12.8)	24.8 (25.2)	3.17 %	83.87 %	218.7 (21.9)	36.3 (27.3)	6.81 %	91.74 %
5:5	213.8 (20.3)	17.0 (20.1)	1.26 %	76.47 %	215.5 (14.3)	34.1 (30.2)	5.43 %	91.20 %
6:4	206.7 (24.2)	45.2 (63.7)	−2.13 %	91.15 %	208.3 (31.1)	46.7 (96.6)	2.16 %	93.58 %
7:3	214.7 (11.0)	54.7 (36.2)	1.68 %	92.69 %	195.3 (16.7)	60.6 (90.4)	−4.35 %	95.05 %
8:2	195.2 (15.8)	133.7 (156.8)	−8.15 %	97.01 %	199.9 (25.1)	54.9 (66.4)	−1.95 %	94.54 %
9:1	190.8 (23.6)	148.3 (179.4)	−10.64 %	97.30 %	206.7 (23.2)	117.5 (82.1)	1.40 %	97.45 %

Table 6
Results of ablation study.

Scenario	State adjustment	A3C			LLM4A3C		
		Makespan (std)	Total tardiness (std)	Objective (std)	Makespan (std)	Total tardiness (std)	Objective (std)
S2	Add $O_{rem}(t)$	224.4 (26.1)	49.4 (40.6)	138.1 (17.3)	211.1 (12.2)	4.0 (9.9)	107.8 (5.0)
	Delete $D_{cur}(t)$	186.2 (8.8)	62.1 (28.8)	122.6 (14.9)			
	Simultaneously add $\tau_{late}(t)$ and delete $D_{cur}(t)$	224.5 (12.3)	58.2 (63.7)	141.3 (35.7)			
S3	Add $M_{idle}(t)$	230.5 (10.6)	18.6 (22.6)	127.6 (11.7)	203.8 (26.5)	3.0 (6.0)	103.4 (14.5)
	Delete $P_{\min}(t)$	210.7 (39.7)	35.3 (23.8)	129.9 (15.2)			
	Simultaneously add $M_{idle}(t)$、$M_{ut}(t)$ and delete $P_{\min}(t)$	228.5 (22.3)	56.2 (53.2)	131.9 (17.2)			

in prior studies [44–46], the proposed methods achieved an average deviation of 2.7 % percent across all instances. This indicates a relatively small performance gap. Overall, the proposed approach demonstrates greater consistency in solution quality and capability compared with the referenced method.

4.2. Experimental results with various scenarios

The experiments in this section are designed to validate the effectiveness of the proposed LLM4A3C method under various scheduling scenarios. The primary objective is to examine whether the dynamic adjustment of state and reward definitions, guided by the LLM, can lead to improved scheduling outcomes. LLM4A3C is compared with the general A3C approach, and their performances are evaluated by makespan, total tardiness, and the weighted objective value to demonstrate the advantage of integrating LLM-driven definition refinement. In addition, DQN and LLM4DQN are included as benchmarks to examine whether the refinement mechanism also benefits value-based methods.

According to the experimental results presented in Table 4, A3C demonstrates comparable overall performance to LLM4A3C in scenario S1. The moving average trends of cumulative rewards, makespan, and total tardiness are shown in Figs. 6-8. An observation can be made that while the objective values obtained by A3C and LLM4A3C are similar, the cumulative rewards differ. This difference is mainly caused by the inclusion of a potential-based shaping term in the reward design of LLM4A3C, which introduces structural changes in reward calculation between the two methods and limits direct comparability. The results of DQN and LLM4DQN also show similar patterns. Indeed, the trajectories of cumulative reward further indicate that all of these algorithms are able to converge across different problem sizes.

Across scenarios S2 and S3, A3C exhibits reduced learning stability under fixed state representations and reward definitions, whereas LLM4A3C attains stable learning dynamics and consistent convergence in all tested settings, indicating stronger generalization under changing conditions. Table 4 further shows that LLM4A3C achieves a lower average makespan and total tardiness than A3C in every scenario. The results also indicate that the advantages of LLM-driven approaches become more pronounced as the problem size increases. In larger instances, such as 30-job and 45-job scenarios, both LLM4A3C and LLM4DQN demonstrate greater reductions in objective values compared with their baseline RL algorithms. This improvement can be attributed to the capacity of the LLM-guided mechanism to dynamically refine state and reward definitions. When the solution space expands with more jobs and operations, the LLM-driven refinement allows the agent to adaptively emphasize more informative features of the evolving environment and to adjust reward signals according to performance trends. Overall, the weighted objective of LLMA3C is consistently reduced relative to A3C, with improvements ranging from 2.65 % to 17.26 %, and the largest gains occur under the environmental changes in S2 and S3. A similar improvement is observed for LLM4DQN over DQN across job counts and scenarios with average improvements 9.65 %. In terms of training duration, LLM-driven methods require slightly higher computational costs than baseline algorithms due to the additional generation and evaluation of state and reward refinements. Nevertheless, the increase remains moderate, and in the case of LLM4A3C the training time advantage of the underlying A3C structure ensures a favorable trade-off between efficiency and solution quality.

4.3. Sensitivity analysis and ablation study

Given the suboptimal performance of the A3C method under scenarios S2 and S3, this section presents a combined sensitivity analysis and ablation study to evaluate how variations in reward function design and state representation affect the overall scheduling performance. The objective is to explore whether appropriate manual adjustments can narrow the performance gap between A3C and the proposed LLM4A3C method. According to the sensitivity analysis under a total of 15 job counts presented in Table 5, various weight ratios of makespan and total tardiness were tested in the weighted objective function. In scenarios S2 and S3, the 5:5 weighting scheme exhibited relatively stable performance and achieved a more balanced trade-off between objectives, yielding generally better overall effectiveness compared to other weighting configurations. In contrast, other weighting configurations often led to imbalanced solutions due to excessive emphasis on a single performance objective. The experimental results also indicate that regardless of weight adjustments, a significant gap remains between A3C and LLM4A3C in terms of total tardiness. Although some settings obtain a slightly better makespan, the weighted objective values were unable to match the the results achieved by the LLM4A3C method. The results also indicate the LLM-guided adaptive refinement in both reward function design and state definition.

The ablation study presented in Table 6 further explored the impact of state definition on learning effectiveness. This analysis aims to determine whether the inferior performance of A3C in scenarios S2 and S3 can be attributed to the absence of critical information or the inclusion of redundant features within the state definition, both of which may impede effective policy learning. In scenario S2, the first modification involved augmenting the state with the average processing time for each operation of unfinished jobs, denoted as $O_{rem}(t)$ shown in Eq. (25), aiming to provide the agent with more detailed information regarding overall workload. However, the results show that this modification failed to yield performance improvement. Subsequently, the due date of the current job $D_{cur}(t)$ was removed in an effort to simplify the state and emphasize more relevant features. While this change led to a marginal improvement in makespan, it had limited impact on reducing total tardiness. To further explore the role of temporal urgency, another tardiness indicator $\tau_{late}(t)$ shown in Eq. (26) was added in the state representation, to calculate the tardiness of jobs are not yet completed but are already late at time t. The experiment adds $\tau_{late}(t)$ while removing $D_{cur}(t)$. Nevertheless, this combination resulted in further degradation of performance, indicating that these manual modifications were ineffective in improving the weighted objective.

In scenario S3, the number of idle machines $M_{idle}(t)$ was first introduced to enhance the agent's awareness of machine utilization. Although this helped reduce total tardiness, the performance remains suboptimal. Next, the shortest average processing time of the machine for the current job $P_{\min}(t)$ state was removed to eliminate redundant information, leading to a slight improvement in makespan but no significant overall benefit. Finally, a combined approach was adopted by adding both the number of idle machines and the machine utilization rate $M_{ut}(t)$, while removing the shortest average processing time of the machine for the current job, in order to better align the state with the operational characteristics of S3. However, this configuration led to results similar to those observed in S2, with a further decline in the weighted objective value. These findings further highlight the advantages of the LLM-guided state representation approach.

Based on the results in Tables 5 and 6, although tuning of reward and state definitions may lead to improved performance on individual metrics, it often introduces trade-offs and conflicts between these metrics, making global optimization of the weighted objective difficult to achieve. In addition, these experiments require considerable computational time and manual effort for parameter tuning and feature design. In contrast, the proposed LLM4A3C method, supported by the reasoning and generalization capabilities of LLM, is able to adaptively refine state definitions and reward functions based on environmental feedback to improve learning efficiency and reducing the need for manual intervention in dynamic FJSP environments.

$$O_{rem}(t) = \frac{1}{N_{rem}(t) \times J} \sum_{i=1}^{n} \frac{\sum_{k \in M_{ij}} P_{ijk}}{|M_{ij}|} \tag{25}$$

Table 7
Comparison of sample efficiency, convergence rate and correlation coefficient.

Scenario	Sample efficiency		Convergence rate		Correlation coefficient	
	A3C	LLM4A3C	A3C	LLM4A3C	A3C	LLM4A3C
S1	0.41 (0.32)	0.45 (0.28)	0.46 (0.05)	0.54 (0.05)	0.45 (0.06)	0.60 (0.05)
S2	0.44 (0.10)	0.50 (0.12)	0.40 (0.17)	0.76 (0.06)	0.23 (0.09)	0.41 (0.06)
S3	0.16 (0.02)	0.55 (0.37)	0.42 (0.17)	0.63 (0.12)	0.12 (0.05)	0.31 (0.01)

$$\tau_{tard}(t) = \sum_{i=1}^{n} (\max(0, c_i - d_i) | \delta_i(t) = 1) \tag{26}$$

4.4. Learning efficiency and effectiveness analysis

Three metrics are employed to further evaluate the learning efficiency and auxiliary benefits of the proposed method [19]. First, the sample efficiency shown in Eq. (27) is used to assess the method's ability to learn effective policies under limited interaction, reflecting how well it utilizes training samples. Here, $T_{80\%}$ denotes the number of training iterations required to reach 80% of the maximum return, and *steps per iteration* is the number of environment interaction steps taken in each iteration. Second, Eq. (28) represents the convergence threshold B_k, which we adopt as a practical indicator to determine whether the reward signal has stabilized. When the observed change in reward falls below this threshold, it suggests that the learning process has entered a stable region and can be considered converged. Where T denotes the episode length, $||D||$ is the dimensionality of the reward representation, λ is the regularization coefficient, φ is the confidence level parameter, z denotes the number of training iterations. To detect convergence, we evaluate the change in rewards between consecutive training iterations. Convergence is considered to have occurred at the first iteration z that satisfies the following condition: $|r_{z+1} - r_z| < B_z$, where r_z denotes the reward obtained at iteration. When the reward change falls below this bound, the learning process is regarded as having entered a stable region, indicating convergence. Finally, to assess the effectiveness of LLM-generated content in supporting RL strategies, the correlation coefficient defined in Eq. (29) is employed to analyze the relationship between the LLM-derived features used to compute the rewards generated based on potential function X and the actual rewards Y. Here, $\overline{X}$ and $\overline{Y}$ denote the sample means of X and Y. This analysis aims to verify whether the LLM is capable of producing informative and relevant content that contributes positively to the training performance of the A3C algorithm.

According to the results under 15 job counts presented in Table 7, LLM4A3C demonstrates significantly improved sample efficiency across all three scenarios, consistently identifying optimal policies more rapidly than A3C. The performance improvement reaches up to 3.4 times in specific case. In terms of convergence speed, LLM4A3C also stabilizes the learned policy faster, with a maximum improvement of approximately 1.9 times. To further evaluate the effectiveness of the LLM-generated content, the correlation coefficient is employed as a metric to quantify the relationship between generated guidance and actual learning outcomes. Specifically, the analysis compares the correlation between the rewards derived from A3C's original state definition and the actual received rewards, against the correlation between the actual rewards in LLM4A3C and the potential function computed from the LLM-generated state variables and sub-goals. The results show that LLM4A3C achieves consistently higher correlation coefficients across all scenarios, with particularly notable improvements observed in scenarios S2 and S3, which also indicate the state and reward definitions generated by LLM can effectively align with the agent's learning process.

$$\textit{Sample Efficiency} = T_{80\%} \times \textit{steps per iteration} \tag{27}$$

$$B_k = \sqrt{\frac{1}{4} T ||D|| \log\left(\frac{1 + zT^2/\lambda}{\varphi/10}\right)} + \sqrt{\lambda ||D||} \tag{28}$$

$$corr(X, Y) = \frac{\sum (X_i - \overline{X})(Y_i - \overline{Y})}{\sqrt{\sum (X_i - \overline{X})^2} \times \sqrt{\sum (Y_i - \overline{Y})^2}} \tag{29}$$

5. Conclusion

This study presents LLM4A3C, a LLM4RL framework that integrates LLM with A3C algorithm to solve the FJSP. The proposed framework introduces a semantic-driven mechanism that enables the dynamic redefinition of state representations and reward functions within the RL loop, based on training feedback interpreted by the LLM. Through iterative generation of adaptive representations, the RL agent can effectively respond to variations in production environments. Experimental results across diverse job sizes and scheduling scenarios indicate that LLM4A3C significantly improves scheduling performance compared to the baseline A3C. The proposed framework demonstrates reductions in both makespan and total tardiness, while also improving sample efficiency and accelerating convergence during training. Furthermore, the ablation study demonstrates that manually adding or removing individual state features does not lead to comparable performance gains, underscoring the limitations of static feature engineering in dynamic scheduling environments. Sensitivity analysis further shows that LLM4A3C consistently outperforms A3C across different objective weight settings. In addition, correlation analysis indicates that reward signals derived from LLM-generated state representations and sub-goals exhibit stronger alignment with actual return trends and validate relevance and effectiveness in guiding policy learning. Compared with the prior studies of LLM4RL, this study presents the application and validation in FJSP, which represents a complex combinatorial optimization problem. Moreover, while most existing studies apply LLMs mainly for reward shaping, the proposed framework introduces a dual refinement mechanism that simultaneously redefines both state representations and reward functions in a closed-loop interaction with DRL. Furthermore, by integrating a greedy policy, the framework dynamically selects and adjusts new state variables and sub-goals according to the performance feedback of the previous DRL training cycle.

Nevertheless, several limitations should be acknowledged. The integration of LLMs introduces additional computational overhead, and the training period may increase further with larger instances. The framework also relies on the quality and consistency of LLM outputs, making it sensitive to prompt design. Moreover, the current experiments validate the framework only on the FJSP and do not consider dedicated constraints such as auxiliary resources or transportation times. Future research can extend the use of LLMs to support policy generation or action selection, and further explore integration with other RL designs to enhance the generalization capability of LLM-driven learning design.

CRediT authorship contribution statement

Tzu-Yen Hong: Writing – review & editing, Validation, Methodology, Investigation, Funding acquisition, Conceptualization. **Kuan-Han Li:** Writing – original draft, Validation, Software, Formal analysis.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research is supported by National Science and Technology Council, Taiwan (NSTC 113-2221-E-027-111; NSTC 114-2221-E-027-098-MY2).

Data availability

Data will be made available on request.

References

[1] K.Z. Gao, P.N. Suganthan, T.J. Chua, C.S. Chong, T.X. Cai, Q.K. Pan, A two-stage artificial bee colony algorithm scheduling flexible job-shop scheduling problem with new job insertion, Expert Syst. Appl. 42 (21) (2015) 7652–7663.

[2] M.R. Garey, D.S. Johnson, R. Sethi, The complexity of flowshop and jobshop scheduling, Math. Oper. Res. 1 (2) (1976) 117–129.

[3] K. Gao, F. Yang, M. Zhou, Q. Pan, P.N. Suganthan, Flexible job-shop rescheduling for new job insertion by using discrete Jaya algorithm, IEEE Trans. Cybern. 49 (5) (2018) 1944–1955.

[4] Y. Li, W. Huang, R. Wu, K. Guo, An improved artificial bee colony algorithm for solving multi-objective low-carbon flexible job shop scheduling problem, Appl. Soft Comput. 95 (2020) 106544.

[5] J. Lin, Backtracking search based hyper-heuristic for the flexible job-shop scheduling problem with fuzzy processing time, Eng. Appl. Artif. Intell. 77 (2019) 186–196.

[6] D. Lei, M. Li, L. Wang, A two-phase meta-heuristic for multiobjective flexible job shop scheduling problem with total energy consumption threshold, IEEE Trans. Cybern. 49 (3) (2018) 1097–1109.

[7] J. Chang, D. Yu, Y. Hu, W. He, H. Yu, Deep reinforcement learning for dynamic flexible job shop scheduling with random job arrival, Processes 10 (4) (2022) 760.

[8] Y.R. Shiue, K.C. Lee, C.T. Su, Real-time scheduling for a smart factory using a reinforcement learning approach, Comput. Ind. Eng. 125 (2018) 604–614.

[9] Y.F. Wang, Adaptive job shop scheduling strategy based on weighted Q-learning algorithm, J. Intell. Manuf. 31 (2) (2020) 417–432.

[10] H. Li, R. Cai, N. Liu, X. Lin, Y. Wang, Deep reinforcement learning: Algorithm, applications, and ultra-low-power implementation, Nano Commun. Netw. 16 (2018) 81–90.

[11] Li, Y. (2017). Deep Reinforcement Learning: An Overview. arXiv preprint arXiv: 1701.07274.

[12] Y. Du, J. Li, C. Li, P. Duan, A reinforcement learning approach for flexible job shop scheduling problem with crane transportation and setup times, IEEE Trans. Neural Netw. Learn. Syst. 35 (4) (2022) 5695–5709.

[13] L. Zhang, Y. Feng, Q. Xiao, Y. Xu, D. Li, D. Yang, Z. Yang, Deep reinforcement learning for dynamic flexible job shop scheduling problem considering variable processing times, J. Manuf. Syst. 71 (2023) 257–273.

[14] W. Zhang, F. Zhao, Y. Li, C. Du, X. Feng, X. Mei, A novel collaborative agent reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for flexible job shop scheduling problem, J. Manuf. Syst. 74 (2024) 329–345.

[15] Z. Zhao, D. Tang, H. Zhu, Z. Zhang, K. Chen, C. Liu, Y. Ji, A large language model-based multi-agent manufacturing system for intelligent shopfloor, arXiv Prepr. arXiv 2405 (2024) 16887.

[16] Y. Cao, H. Zhao, Y. Cheng, T. Shu, Y. Chen, G. Liu, Y. Li, Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods, IEEE Trans. Neural Netw. Learn. Syst. (2024) 1–21.

[17] Zhang, F., Li, J., Li, Y.C., Zhang, Z., Yu, Y., & Ye, D. (2024). Improving sample efficiency of reinforcement learning with background knowledge from large language Models. arXiv preprint arXiv:2407.03964.

[18] Sun, S., Liu, R., Lyu, J., Yang, J.W., Zhang, L., & Li, X. (2024). A large language model-driven reward design framework via dynamic feedback for reinforcement learning. arXiv preprint arXiv:2410.14660.

[19] Y. Qu, Y. Jiang, B. Wang, Y. Mao, C. Wang, C. Liu, X. Ji, Latent reward: LLM-empowered credit assignment in episodic reinforcement learning, arXiv Prepr. arXiv 2412 (2024) 11120.

[20] V. Mnih, A.P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, K. Kavukcuoglu, Asynchronous methods for deep reinforcement learning, Proc. 33rd Int. Conf. Mach. Learn. 48 (2016, June) 1928–1937.

[21] K.Z. Gao, P.N. Suganthan, Q.K. Pan, M.F. Tasgetiren, A. Sadollah, Artificial bee colony algorithm for scheduling and rescheduling fuzzy flexible job shop problem with new job insertion, Knowl. Based Syst. 109 (2016) 1–16.

[22] M. Zandieh, A.R. Khatami, S.H.A. Rahmati, Flexible job shop scheduling under condition-based maintenance: improved version of imperialist competitive algorithm, Appl. soft Comput. 58 (2017) 449–464.

[23] Y. Li, W. Huang, R. Wu, K. Guo, An improved artificial bee colony algorithm for solving multi-objective low-carbon flexible job shop scheduling problem, Appl. Soft Comput. 95 (2020) 106544.

[24] Z. Guo, L. Wei, J. Zhang, Z. Hu, H. Sun, H. Che, Multi-level guided evolution algorithm for solving fuzzy flexible job shop problem, Appl. Soft Comput. 163 (2024) 111932.

[25] J. Li, W. Shi, N. Zhang, X. Shen, Delay-aware VNF scheduling: a reinforcement learning approach with variable action set, IEEE Trans. Cogn. Commun. Netw. 7 (1) (2020) 304–318.

[26] K. Alexopoulos, N. Nikolakis, E. Bakopoulos, V. Siatras, P. Mavrothalassitis, Machine learning agents augmented by digital twinning for smart production scheduling, IFACPap 56 (2) (2023) 2963–2968.

[27] J.C. Serrano-Ruiz, J. Mula, R. Poler, Job shop smart manufacturing scheduling by deep reinforcement learning, *Journal of Industrial Information Integration* 38 (2024) 100582.

[28] R. Naimi, M. Nouiri, O. Cardin, A Q-learning rescheduling approach to the flexible job shop problem combining energy and productivity objectives, Sustainability 13 (23) (2021) 13016.

[29] S. Luo, Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning, Appl. Soft Comput. 91 (2020) 106208.

[30] D. Johnson, G. Chen, Y. Lu, Multi-agent reinforcement learning for real-time dynamic production scheduling in a robot assembly cell, IEEE Robot. Autom. Lett. 7 (3) (2022) 7684–7691.

[31] Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., … & Wierstra, D. (2015). Continuous control with deep reinforcement learning. arXiv preprint arXiv: 1509.02971.

[32] L. Ding, Z. Guan, M. Rauf, L. Yue, Multi-policy deep reinforcement learning for multi-objective multiplicity flexible job shop scheduling, Swarm Evolut. Comput. 87 (2024) 101550.

[33] W. Gu, S. Liu, Z. Guo, M. Yuan, F. Pei, Dynamic scheduling mechanism for intelligent workshop with deep reinforcement learning method based on multi-agent system architecture, Comput. Ind. Eng. 191 (2024) 110155.

[34] K.H. Ho, J.Y. Cheng, J.H. Wu, F. Chiang, Y.C. Chen, Y.Y. Wu, I.C. Wu, Residual scheduling: a new reinforcement learning approach to solving job shop scheduling problem, IEEE Access 12 (2024) 14703–14718.

[35] R. Wang, G. Wang, J. Sun, F. Deng, J. Chen, Flexible job shop scheduling via dual attention network-based reinforcement learning, IEEE Trans. Neural Netw. Learn. Syst. 35 (3) (2023) 3091–3102.

[36] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, Unifying large language models and knowledge graphs: a roadmap, IEEE Trans. Knowl. Data Eng. 36 (7) (2024) 3580–3599.

[37] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, Q. Wang, Software testing with large language models: survey, landscape, and vision, IEEE Trans. Softw. Eng. 50 (4) (2024) 911–936.

[38] M. Pternea, P. Singh, A. Chakraborty, Y. Oruganti, M. Milletari, S. Bapat, K. Jiang, The rl/llm taxonomy tree: reviewing synergies between reinforcement learning and large language models, J. Artif. Intell. Res. 80 (2024) 1525–1573.

[39] E. Pignatelli, J. Ferret, T. Rockäschel, E. Grefenstette, D. Paglieri, S. Coward, L. Toni, Assessing the zero-shot capabilities of LLMs for action evaluation in RL, arXiv Prepr. arXiv 2409 (2024) 12798.

[40] D.A. Nguyen, R.K. Mohan, V. Yang, P.S. Akash, K.C.C. Chang, RL-based query rewriting with distilled LLM for online E-commerce systems, arXiv Prepr. arXiv 2501 (2025) 18056.

[41] Tu, S., Sun, J., Zhang, Q., Lan, X., & Zhao, D. (2024). Online Preference-based Reinforcement Learning with Self-augmented Feedback from Large Language Model. arXiv preprint arXiv:2412.16878.

[42] K. Karine, B.M. Marlin, Combining LLM decision and RL action selection to improve RL policy for adaptive interventions, arXiv Prepr. arXiv 2501 (2025) 06980.

[43] K. Lei, P. Guo, W. Zhao, Y. Wang, L. Qian, X. Meng, L. Tang, A multi-action deep reinforcement learning framework for flexible Job-shop scheduling problem, Expert Syst. Appl. 205 (2022) 117796.

[44] S. Dauzère-Pérès, J. Paulli, An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search, Ann. Oper. Res. 70 (0) (1997) 281–306.

[45] P. Brucker, B. Jurisch, B. Sievers, A branch and bound algorithm for the job-shop scheduling problem, Discret. Appl. Math. 49 (1-3) (1994) 107–127.

[46] M. Mastrolilli, L.M. Gambardella, Effective neighbourhood functions for the flexible job shop problem, J. Sched. 3 (1) (2000) 3–20.